

CSL Sleepy Router

Documentazione tecnica completa delle modifiche rispetto alla repo standard di OpenThread — architettura, funzioni, percorso dati, problemi noti.

1. Obiettivo del progetto

In Thread standard un Router (FTD) resta sempre acceso (`RxOnWhenIdle = true`): è lui che fa da relay per i suoi child, quindi deve poter ricevere in ogni momento. Il CSL (Coordinated Sampled Listening, IEEE 802.15.4/Thread 1.2+) esiste per permettere il contrario — un dispositivo sleepy che ascolta solo in finestre periodiche — ma di serie è pensato per un solo verso: il *Child* dorme, il *Parent* (sempre acceso) accoda e trasmette nella finestra giusta.

Questo fork ha due obiettivi complementari, sviluppati in due fasi:

- **Fase 1 — il Router dorme.** Un Router può entrare in modalità "Sleepy Router": pubblicizza un periodo CSL come farebbe un normale Parent verso un child sleepy, pur restando Router a tutti gli effetti nel protocollo (ruolo, eleggibilità, routing invariati).
- **Fase 2 — il Child parla al Router sleepy.** Chi è agganciato a un Router ora sleepy deve smettere di mandargli pacchetti in trasmissione diretta (che fallirebbe, il parent non ascolta più sempre) e deve invece riusare la stessa infrastruttura di invio indiretto già usata per i child sleepy — ma questa volta per parlare "verso l'alto", al proprio parent.

2. Terminologia rapida

- **CSL Receiver** (`OPENTHREAD_CONFIG_MAC_CSL_RECEIVER_ENABLE`): il dispositivo è quello che dorme e si sveglia a finestre. Di serie: i child sleepy.
- **CSL Transmitter** (`OPENTHREAD_CONFIG_MAC_CSL_TRANSMITTER_ENABLE`): il dispositivo accoda messaggi e li trasmette nella finestra di ascolto altrui. Di serie: il Parent verso i suoi child sleepy. In questo fork: *anche* un Child verso il proprio Parent sleepy.
- **IndirectSender**: sottosistema che gestisce la coda dei messaggi "indiretti" (non trasmessi subito, ma accodati per un destinatario sleepy) e la loro preparazione/finalizzazione.
- **CsITxScheduler**: decide *quando* trasmettere CSL a chi, tra tutti i vicini CSL-sincronizzati candidati, e coordina con Mac il timing preciso.
- **DataPollHandler**: gestisce i poll classici (802.15.4) dei child non-CSL; non toccato in profondità da questo lavoro.
- **CslNeighbor**: tipo che rappresenta "un vicino con cui faccio CSL"; di serie coincide sempre con Child. Parent *non* eredita da CslNeighbor (per evitare eredità a diamante, dato che eredita già da Router) — compone invece direttamente i tre mixin NeighborInfo di cui sopra.

3. Modifiche per sottosistema

3.1 API pubblica e CLI

`include/openthread/thread.h` aggiunge `otThreadSetSleepyRouterMode()` e `otThreadIsSleepyRouterMode()`, implementate in `thread_api.cpp` come thin wrapper su `Mle::SetSleepyRouterMode()/IsSleepyRouterMode()`. `src/cli/cli.cpp` espone il comando `sleepyrouter enable|disable` (registrato solo per `OPENTHREAD_FTD`, coerente col fatto che solo un Router può dormire in questo modo).

3.2 Configurazione

- `config/mac.h`: `OPENTHREAD_CONFIG_MAC_DEFAULT_MAX_FRAME_RETRIES_DIRECT` 15→5 (retry diretti più aggressivi/veloci nei test).
- `config/mle.h`: `OPENTHREAD_CONFIG_MLE_CHILD_TIMEOUT_DEFAULT` 240→20 secondi (timeout child più corto, utile per iterare velocemente nei test OTNS ma non necessariamente adatto a una rete reale).

3.3 MAC layer — Sleepy Router e sincronizzazione del Parent

Mac guadagna mSleepyRouterPeriod (0 = disabilitato) con SetSleepyRouterCslPeriod()/GetSleepyRouterCslPeriod() e la costante kDefaultSleepyRouterCslPeriod = 1000 (in unità di 10 simboli).

In Mac::BeginTransmit(), se il router è sleepy e il frame in uscita contiene già un IE CSL, ne viene ricalcolata al volo fase e periodo in base al clock radio corrente — è così che un Router sleepy "si racconta" ai suoi vicini nei frame che manda comunque (broadcast periodici, risposte dirette), esattamente come farebbe un Parent normale con i suoi child.

Mac::ProcessCsl() (che riceve un frame in ingresso con IE CSL) prima gestiva solo il caso "il mittente è un mio child" (tabella childTable, solo FTD). È stata aggiunta una seconda diramazione, sempre presente: se il mittente non è un child, si controlla se corrisponde al proprio Parent (per MAC esteso o RLOC16) — in tal caso si aggiornano SetCslPeriod/SetCslPhase/SetCslSynchronized(true)/SetCslLastHeard/SetLastRxTimestamp sul Parent. È il punto in cui il Child "impara" quando il proprio Router sleepy sarà in ascolto.

mac_frame.hpp/.cpp: le funzioni di lettura/scrittura dell'IE CSL (SetCslIe, HasCslIe, GetCslIe) erano gated dietro OPENTHREAD_CONFIG_MAC_CSL_RECEIVER_ENABLE — vero di serie solo per dispositivi sleepy "classici". Ora sono disponibili anche con il solo OPENTHREAD_FTD, perché un Router sleepy deve poter scrivere l'IE CSL nei propri frame pur non essendo un CSL Receiver in senso classico.

message_framer.cpp / PrepareMacHeaders(): se la destinazione è broadcast e il router è sleepy, marca il frame per includere l'IE CSL (versione 2015). Per un vicino nella childTable, oltre al caso CSL-verso-child già esistente, aggiunge lo stesso trattamento quando il router stesso è sleepy (l'IE CSL va nei frame anche verso i child "normali", non solo in risposta ai loro poll).

3.4 MLE — attivazione della modalità e un compromesso rischioso

Mle::SetSleepyRouterMode(bool): se il nuovo stato è diverso da quello corrente, aggiorna mIsSleepyRouter e (solo FTD) collega/scollega Mac::SetSleepyRouterCslPeriod() al valore di default o a 0. Mle::SetDeviceMode() normalmente rifiuta un DeviceMode non "valido" — e per la libreria un FTD deve avere RxOnWhenIdle = true per essere valido (vedi §5.2). La modifica aggiunge un'eccezione: se IsSleepyRouterMode() è attivo, quel controllo viene saltato.

Attenzione — modifica rischiosa, marcata esplicitamente come "HACK" nel codice: in Mle::RetxTracker::RetryInfo::DetachIfMaxAttemptsReached(), la chiamata originale a aMle.BecomeDetached() (che normalmente fa ri-aggianciare il dispositivo quando i retry MLE verso il parent sono esauriti) è stata **commentata** e sostituita con un semplice log ("HACK: Max attempts reached, detach blocked"). Il device non si stacca più mai automaticamente per questa causa, qualunque sia la situazione reale. È stato introdotto per non far vedere "detach" spuri durante i test con la pipeline CSL ancora incompleta, ma è un cambiamento di comportamento significativo che nasconde un meccanismo di recovery standard di Thread: da rivalutare prima di qualunque uso oltre il testing locale.

3.5 Router / Parent — comporre il Parent come CSL neighbor

class Parent : public Router ora eredita anche, sotto OPENTHREAD_CONFIG_MAC_CSL_TRANSMITTER_ENABLE, da CslTxScheduler::NeighborInfo, DataPollHandler::NeighborInfo e IndirectSender::NeighborInfo — gli stessi tre mixin di cui già dispone Child. Questo dà al Parent, gratis, tutti i campi/metodi (periodo, fase, contatori di retry, messaggio indiretto corrente, offset di frammento...) di cui la pipeline di invio indiretto ha bisogno, senza duplicare codice.

Caveat non banale, da tenere a mente: Parent ha *anche* un proprio set "storico" di campi CSL (mCslPeriod, mCslPhase, mCslSynchronized) dichiarato direttamente nella classe sotto #if OPENTHREAD_FTD, risalente alla fase iniziale del lavoro (spostamento dei campi CSL da Router a Parent), con relativi GetCslPeriod()/SetCslPeriod()/ecc. propri. Su un build MTD (OPENTHREAD_FTD == 0) questo blocco semplicemente non esiste, e le uniche versioni attive sono quelle ereditate da CslTxScheduler::NeighborInfo — nessun problema. Ma su un build **FTD** con CSL_TRANSMITTER_ENABLE attivo (il caso di default, dato che quella macro è true da Thread 1.2 in su), *entrambi* gli insiemi esistono contemporaneamente: per le regole di visibilità del C++, i metodi dichiarati direttamente in Parent "nascondono" (non entrano in conflitto/ambiguità, semplicemente

vincono) quelli ereditati con lo stesso nome. Il codice compila e funziona in modo coerente perché sia il codice storico sia il nuovo codice CSL chiamano gli accessor allo stesso modo (accesso non qualificato) e finiscono quindi per usare sempre gli stessi campi "vincenti" — ma la seconda copia di storage (quella ereditata) resta comunque allocata nell'oggetto, inutilizzata, ed è un potenziale trabocchetto se in futuro qualcuno accede esplicitamente alla versione qualificata (es. `parent.CslTxScheduler::NeighborInfo::GetCslPeriod()`) aspettandosi lo stesso valore. Da ripulire (tenere solo una delle due copie) in un passaggio successivo.

3.6 Message — un bit per il Parent

`common/message.hpp` aggiunge `IsPendingForParent()`, `SetPendingForParent()`, `ClearPendingForParent()`. È l'equivalente, per il Parent, del `GetIndirectTxChildMask()` usato per i Child: lì serve una bitmap perché un messaggio può essere pendente per *più* child contemporaneamente, qui basta un solo bit perché un messaggio può avere *un solo* parent.

3.7 IndirectSender — la pipeline di invio verso il Parent

Nuove funzioni, tutte sotto `OPENTHREAD_CONFIG_MAC_CSL_TRANSMITTER_ENABLE` (quindi disponibili sia su FTD sia su MTD), ciascuna specchio di una funzione già esistente per i Child ma adattata al fatto che esiste un solo Parent, non una tabella:

Funzione / campo	Descrizione
<code>AddMessageForSleepyParent(Message&, Parent&)</code>	Punto d'ingresso da <code>MeshForwarder::SendMessage()</code> . Marca il messaggio pendente per il parent e chiede l'aggiornamento (<code>RequestMessageUpdate</code>). Contiene un assert sull'invariante "il parent è davvero CSL-sincronizzato", coerente col controllo già fatto dal chiamante.
<code>RemoveMessageFromSleepyParent(Message&)</code>	Toglie il flag pendente-per-parent da un messaggio (es. quando va rimosso prima di essere spedito) e richiama l'aggiornamento.
<code>FindQueuedMessageForSleepyParent(Parent&, MessageChecker)</code>	Scansiona <code>MeshForwarder::mSendQueue</code> e ritorna il primo messaggio con <code>IsPendingForParent()</code> che soddisfa il predicato passato (tipicamente <code>AcceptAnyMessage</code> , mantenuto condiviso e non ristretto a FTD apposta, per lasciare al Parent la stessa flessibilità di filtraggio per categoria che ha un Child).
<code>UpdateIndirectMessage(Parent&)</code>	Cerca il prossimo messaggio candidato e lo assegna come "messaggio indiretto corrente" del parent (<code>SetIndirectMessage</code>), azzera l'offset di frammento, richiama <code>CslTxScheduler::Update()</code> .
<code>RequestMessageUpdate(Parent&)</code>	Punto centrale chiamato ogni volta che la coda potrebbe essere cambiata per il parent: se il messaggio corrente e quello trovato ora coincidono non fa nulla; altrimenti (e solo se il fragment offset è 0) richiama <code>UpdateIndirectMessage</code> . Lo scenario "il cursore punta a un messaggio il cui <code>IsPendingForParent()</code> è nel frattempo diventato falso" (il "purge", equivalente al Block A della versione Child) è stato volutamente rimandato — non gestito ora.
<code>PrepareFrameForParent(Mac::TxFrame&, FrameContext&, Parent&)</code>	Costruisce davvero il frame MAC: se non c'è messaggio (o è di tipo Supervision) prepara un frame vuoto (probe), altrimenti un frame IPv6 vero a partire dall'offset di frammento corrente.
<code>HandleSentFrameToCslParent(const Mac::TxFrame&, const FrameContext&, Error, Parent&)</code>	Finalizza l'esito dell'invio: aggiorna successo/fallimento, se il messaggio è concluso lo rimuove dalla coda indiretta del parent (<code>ClearPendingForParent</code> , non più la bitmap dei child), invoca il callback di tx, aggiorna contatori/log, richiama <code>MeshForwarder::RemoveMessageIfNoPendingTx</code> e infine <code>UpdateIndirectMessage</code> .

3.8 CslTxScheduler — schedulare anche il Parent

Nuovo campo `bool mSchedulingParent`: dice se il prossimo invio CSL pianificato è per il Parent (true) o per un Child tramite `mCslTxNeighbor` (false). Il Parent *non* è mai assegnato a `mCslTxNeighbor` (che resta tipizzato `CslNeighbor*`, e Parent non eredita da `CslNeighbor`) — da qui la necessità del flag separato e di un percorso a parte in ciascuna funzione.

Funzione / campo	Descrizione
------------------	-------------

<code>RescheduleCslTx()</code>	Trova il candidato più vicino tra i Child (solo FTD, loop su <code>ChildTable</code>) e — sempre, se sotto <code>CSL_TRANSMITTER_ENABLE</code> — il Parent, se sono un Child, il parent è valido, CSL-sincronizzato e ha un messaggio in coda. Se il Parent vince il confronto imposta <code>mSchedulingParent = true</code> . La richiesta a <code>Mac::RequestCslFrameTransmission()</code> scatta se c'è un <code>bestNeighbor</code> (Child) oppure se <code>mSchedulingParent</code> è vero — condizione facile da dimenticare (bug reale incontrato e corretto durante lo sviluppo).
<code>HandleFrameRequest(Mac::TxFrame&)</code>	Callback da Mac quando serve davvero il prossimo frame da trasmettere. Se <code>mSchedulingParent</code> è vero, prepara il frame per il Parent (via <code>PrepareFrameForParent</code>), calcola il ritardo fino alla prossima finestra CSL (<code>GetNextCslTransmissionDelay</code>) e imposta <code>TxDelay/TxDelayBaseTime</code> sul frame; altrimenti segue l'identico percorso preesistente per i Child.
<code>HandleSentFrame(const Mac::TxFrame&, Error) – 2 parametri</code>	Vero callback da Mac a invio concluso. Azzera <code>mCslTxMessage</code> incondizionatamente (va fatto <i>prima</i> di qualunque uscita anticipata), poi smista a <code>HandleSentFrameToParent</code> o alla versione Child a 3 parametri. In coda, richiama sempre <code>RescheduleCslTx()</code> per far ripartire la valutazione "chi è il prossimo".
<code>HandleSentFrameToParent(const Mac::TxFrame&, Error, Parent&)</code>	Equivalente per il Parent della versione a 3 parametri già esistente per <code>CslNeighbor&</code> : gestisce successo/retry/abbandono in base all'errore, poi (solo se non è un semplice retry/abort) richiama <code>IndirectSender::HandleSentFrameToCslParent</code> .
<code>GetNextCslTransmissionDelay(...)</code>	Firma allargata da <code>const CslNeighbor&</code> a <code>const CslTxScheduler::NeighborInfo&</code> — corpo invariato, ma ora accetta sia un Child sia un Parent, dato che entrambi compongono quel mixin.

3.9 MeshForwarder — instradare verso il Parent invece che diretto

`SendMessage()` (sia in `mesh_forwarder_ftd.cpp` sia in `mesh_forwarder_mtd.cpp`, riscritte separatamente perché il file FTD gestisce anche il fan-out multicast e la tabella dei child, del tutto assente lato MTD) ora, per un messaggio IPv6 unicast: se il ruolo è Child e il parent è valido/CSL-sincronizzato/con periodo > 0 → `AddMessageForSleepyParent`; se il parent non è sincronizzato → trasmissione diretta normale (fallback esplicito, necessario: senza non ci sarebbe alcuna marcatura e il messaggio verrebbe silenziosamente scartato); se non si è affatto Child (Detached su MTD, oppure Router/Leader su FTD) → trasmissione diretta.

`FinalizeMessageIndirectTxToParent(Message&)` (nuova, in `mesh_forwarder.cpp`, chiamata da `FinalizeAndRemoveMessage()`) richiama `IndirectSender::RemoveMessageFromSleepyParent` quando un messaggio viene definitivamente finalizzato, per simmetria con l'equivalente già esistente per i child.

Nota — soluzione "pragmatica" ancora presente come rete di sicurezza: in `MeshForwarder::HandleFrameRequest()` (non quella di `CslTxScheduler`, l'altra, storica) sopravvive ancora il primo fix, più semplice, con cui è iniziato questo lavoro: se un frame diretto è destinato al proprio parent CSL-sincronizzato, gli viene attaccato un `TxDelay` calcolato al volo per farlo coincidere con la prossima finestra, invece di instradarlo davvero attraverso `CslTxScheduler/IndirectSender`. Va rimosso solo dopo aver validato che la pipeline nuova regge da sola in ogni scenario (vedi §6) — per ora resta come backup.

3.10 data_poll_sender.cpp — nota tecnica

In `CalculatePollPeriod()`, ramo `mRetxMode`, il periodo di poll viene sovrascritto con un valore fisso di 25000 ms (25s), indipendentemente dal calcolo che lo precede. Segnalato ma non rimosso: tech debt noto, non necessario per il funzionamento della pipeline CSL.

4. Percorso dati end-to-end (Child → Parent sleepy)

Sequenza completa di un messaggio applicativo (es. un ping ICMPv6) dal momento in cui il Child lo genera fino alla conferma di consegna, quando il Parent è in modalità Sleepy Router:

- `MeshForwarder::SendMessage()` verifica ruolo/stato/sync del parent e chiama `IndirectSender::AddMessageForSleepyParent()` invece di `SetDirectTransmission()`.
- `AddMessageForSleepyParent` marca il messaggio (`SetPendingForParent`) e chiama `RequestMessageUpdate(Parent&)`.
- `RequestMessageUpdate` trova il messaggio (se è nuovo rispetto a quello corrente) e chiama `UpdateIndirectMessage`, che lo assegna al parent e richiama `CsLtxScheduler::Update()`.
- `Update()` (nessun invio già in corso) richiama `RescheduleCsLtx()`, che trova il Parent come miglior candidato, imposta `mSchedulingParent = true` e chiede a Mac di pianificare la trasmissione CSL al tempo giusto (`RequestCsLFrameTransmission`).
- Quando arriva il momento, `Mac::BeginTransmit()` richiama `CsLtxScheduler::HandleFrameRequest()`, che (essendo `mSchedulingParent` vero) chiama `IndirectSender::PrepareFrameForParent()` per costruire il frame reale, poi calcola `TxDelay` per allinearsi alla finestra CSL del parent.
- Il frame parte; sul lato Router, se sleepy, la finestra di ascolto CSL è aperta nel momento giusto e il frame arriva.
- A invio concluso, Mac richiama `CsLtxScheduler::HandleSentFrame()`, che smista a `HandleSentFrameToParent()`: aggiorna contatori di successo/retry, poi chiama `IndirectSender::HandleSentFrameToCsLParent()`, che rimuove il messaggio dalla coda (`ClearPendingForParent`) se completato, e richiama `MeshForwarder::RemoveMessageIfNoPendingTx`.
- In coda, `RescheduleCsLtx()` gira di nuovo per decidere il prossimo invio (Child o Parent) da pianificare.

5. Problemi noti / tech debt

5.1 Packet loss residua sul percorso Child→Parent — indagine in corso, non risolta

Test OTNS: 20 ping Child→Parent, 3 ricevuti (85% di perdita). Analisi dei log: i pacchetti falliti mostrano sempre 4 tentativi CSL consecutivi senza ack ("CSL tx to XXXX failed, attempt N/4") fino all'abbandono; i pacchetti riusciti sono quelli inviati a ridosso di un Advertisement periodico del router (che porta con sé un IE CSL aggiornato).

Ipotesi 1 — canale sbagliato: esclusa. Verificato sia staticamente sia empiricamente. `Parent::Clear()` (router.cpp) fa `ClearAllBytes(*this)`, che azzerava anche i campi ereditati da `CsLtxScheduler::NeighborInfo` incluso `mCsLChannel`; `ProcessCsL()` non lo imposta mai, quindi resta 0 → `HandleFrameRequest` usa sempre `Get<Mac::Mac>().GetPanChannel()`. Aggiunto un `LogInfo` diagnostico che stampa canale usato e PAN channel ad ogni tentativo: su 14 tentativi osservati, **combaciano sempre** (es. `channel=11 panChannel=11`).

Ipotesi 2 — drift di sincronizzazione nel tempo: indebolita, non più l'ipotesi principale. I valori di `txDelay` raccolti sono grandi (fino a 14.6s), ma questo di per sé non è anomalo — `TxDelayBaseTime/TxDelay` vanno sommati per ricostruire un istante assoluto (solo i bit bassi contano, per progetto). Più significativo: il Router calcola la fase CSL in `Mac::BeginTransmit()` come "tempo al prossimo multiplo del periodo contato dal tempo radio assoluto zero" — una griglia fissa e globale, non legata al singolo frame. Se corretto, il calcolo del Child dovrebbe restare valido indipendentemente da quanto tempo è passato dall'ultima sincronizzazione, il che rende il "drift" un colpevole meno probabile di quanto pensato inizialmente.

Sospetto attuale (non ancora verificato): dato che nessuna modifica di questo fork tocca `sub_mac.cpp` — dove vive la vera logica di duty-cycling radio CSL, dietro `OPENTHREAD_CONFIG_MAC_CSL_RECEIVER_ENABLE`, spenta di default e mai attivata per l'FTD — il Router probabilmente non spegne mai davvero la radio a livello di piattaforma. Se così fosse, dovrebbe poter ricevere comunque, sempre: il fatto che invece fallisca puntualmente suggerisce che il problema potrebbe non essere nella logica di protocollo scritta in questo fork, ma nel modo in cui `ot-rf-sim` (il simulatore radio) gestisce concretamente una trasmissione con `TxDelay` impostato (schedulazione radio ritardata) — un'area non ancora indagata. **Non risolto — prossimo problema da affrontare, probabilmente più a fondo di un quick fix.**

5.2 Detach automatico disabilitato

Vedi §3.4: `Mle::RetxTracker::RetryInfo::DetachIfMaxAttemptsReached()` non stacca più il device sui retry MLE esauriti. Utile per non interrompere i test, ma nasconde un vero meccanismo di recovery — da riabilitare o sostituire con qualcosa di più mirato prima di andare oltre il testing locale.

5.3 Doppio set di campi CSL su Parent (build FTD)

Vedi §3.5 — non è un bug attivo (grazie al name-hiding del C++ tutto il codice usa coerentemente la stessa copia), ma è duplicazione di stato pericolosa in caso di accesso qualificato futuro. Da ripulire.

5.4 Scenario "purge" non gestito

In `RequestMessageUpdate(Parent&)` non è gestito il caso in cui il cursore (`aParent.GetIndirectMessage()`) punti a un messaggio il cui `IsPendingForParent()` è nel frattempo diventato falso (equivalente al "Block A" della versione Child). Deciso di rimandarlo esplicitamente durante lo sviluppo.

5.5 Fallback pragmatico ancora attivo

Vedi §3.9 — il `TxDelay` "storico" in `mesh_forwarder.cpp` coesiste con la pipeline nuova. Va rimosso solo dopo aver isolato e confermato che la pipeline nuova funziona da sola end-to-end (compreso il problema di §5.1).

5.6 Nessuna copertura di test automatica

Nessun test Nexus/unit scritto per `CsLTxScheduler/IndirectSender` lato Parent. Tutta la validazione fin qui è manuale via OTNS.

6. Stato di validazione

- Bug iniziali (100% packet loss, versione "pragmatica" pre-refactor): diagnosticati e corretti — verificato 0% loss su OTNS.
- Refactor completo verso la pipeline CSL vera (Parent come CSL neighbor): compila su FTD e MTD dopo diversi cicli di bug-fixing (guardie di macro sbagliate, funzioni duplicate, assert copiati dal Child e non validi per il Parent, `#include` mancante, condizione di trigger dimenticata in `RescheduleCsLTx`, `mIndirectSender.Start()` mai chiamato su MTD — tutti risolti).
- Test end-to-end su OTNS: i messaggi passano davvero attraverso la nuova pipeline (confermato via log mirati), ma con packet loss residua per il problema di sincronizzazione discusso in §5.1 — **non ancora al livello di affidabilità del fix pragmatico originale.**

7. Prossimi passi consigliati

- Indagare e risolvere il drift di sincronizzazione CSL (§5.1) — probabilmente il lavoro più importante rimasto.
- Isolare il fallback pragmatico (§5.5) e confermare che la pipeline nuova regge da sola.
- Ripulire il doppio set di campi CSL su Parent (§5.3).
- Rivalutare l'hack sul detach (§5.2) prima di qualunque uso reale.
- Scrivere test Nexus dedicati (§5.6).
- Gestire lo scenario "purge" (§5.4) se emergono casi reali che lo richiedono.